

Paimon Arrow 模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-arrow

分支版本：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月28日

第1章 模块概述

1.1 模块定位

paimon-arrow 是 Apache Paimon 与 Apache Arrow 之间的桥接模块，负责在 Paimon 内部数据表示（InternalRow、ColumnVector）与 Apache Arrow 的列式内存格式（VectorSchemaRoot、FieldVector）之间进行高效的双向转换。该模块是 Paimon 实现跨语言数据交换（通过 Arrow C Data Interface）和高性能列式数据处理的核心基础组件。

1.2 核心目标

- 类型映射：将 Paimon 的 DataType 体系完整映射到 Arrow FieldType 体系，覆盖全部基本类型和复杂类型（Array、Map、Row、Vector、Variant）
- 写入方向：将 Paimon InternalRow/ColumnVector 数据高效写入 Arrow VectorSchemaRoot
- 读取方向：将 Arrow FieldVector 转换回 Paimon ColumnVector，支持向量化批量读取
- 跨语言交换：通过 Arrow C Data Interface（ArrowArray/ArrowSchema）支持 JNI/Native 跨语言数据传输
- Bundle 集成：与 Paimon 的 BundleRecords/BundleFormatWriter 体系无缝集成

1.3 源码文件一览

文件名	包路径	职责描述
ArrowBundleRecords.java	arrow	VectorSchemaRoot 的 BundleRecords 包装器，支持行迭代
ArrowFieldTypeConversion.java	arrow	Paimon DataType 到 Arrow FieldType 的类型转换访问器
ArrowUtils.java	arrow	Arrow 对象创建工具类（Schema/Vector/Writer/序列化）
Arrow2PaimonVectorConverter.java	converter	Arrow FieldVector 到 Paimon ColumnVector 的转换器
ArrowBatchConverter.java	converter	批量转换基类，管理可复用的 VectorSchemaRoot
ArrowPerRowBatchConverter.java	converter	逐行迭代方式的批量转换器
ArrowVectorizedBatchConverter.java	converter	向量化批量转换器，支持删除向量过滤
ArrowBatchReader.java	reader	从 VectorSchemaRoot 批量读取 Paimon 行的读取器
NativeReader.java	reader	Native JNI 读取器抽象定义

文件名	包路径	职责描述
ArrowCStruct.java	vector	Arrow C Data Interface 结构体缓存
ArrowFormatCWriter.java	vector	支持 C 结构体输出的 Arrow 格式写入器
ArrowFormatWriter.java	vector	InternalRow 到 VectorSchemaRoot 的核心写入器
OneElementFieldVectorGenerator.java	vector	单值重复填充的 FieldVector 生成器
ArrowBundleWriter.java	writer	Arrow Bundle 格式写入器，集成 NativeWriter
ArrowFieldWriter.java	writer	字段级 Arrow 写入器抽象基类
ArrowFieldWriterFactory.java	writer	ArrowFieldWriter 的工厂接口
ArrowFieldWriterFactoryVisitor.java	writer	基于访问者模式创建 ArrowFieldWriterFactory
ArrowFieldWriters.java	writer	全部具体类型的 ArrowFieldWriter 实现（20+ 子类）
NativeWriter.java	writer	Native JNI 写入器抽象定义

第2章 核心架构设计

2.1 整体架构图

paimon-arrow 模块采用四层架构设计，从上层 Bundle 集成层到底层 Native 交互层，每层职责清晰分离：

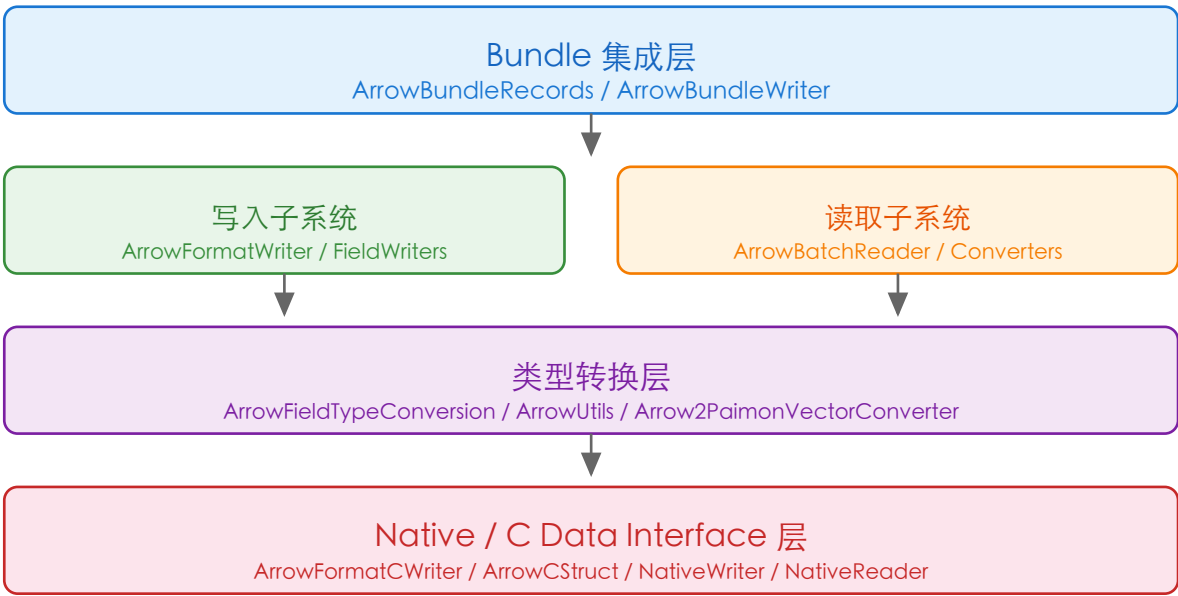


图2-1 paimon-arrow 模块四层架构图

2.2 类继承层次

模块内的核心类继承关系如下：

2.2.1 写入器继承体系

ArrowFieldWriter 是所有字段写入器的抽象基类，ArrowFieldWriters 中包含 20+ 个具体子类：

子类	目标 Arrow Vector	对应 Paimon 类型
StringWriter	VarCharVector	CHAR / VARCHAR
BooleanWriter	BitVector	BOOLEAN
BinaryWriter	VarBinaryVector	BINARY / VARBINARY
DecimalWriter	DecimalVector	DECIMAL
TinyIntWriter	TinyIntVector	TINYINT

子类	目标 Arrow Vector	对应 Paimon 类型
SmallIntWriter	SmallIntVector	SMALLINT
IntWriter	IntVector	INT
BigIntWriter	BigIntVector	BIGINT
FloatWriter	Float4Vector	FLOAT
DoubleWriter	Float8Vector	DOUBLE
DateWriter	DateDayVector	DATE
TimeWriter	TimeMilliVector	TIME
TimestampWriter	TimeStampVector	TIMESTAMP / TIMESTAMP_LTZ
VariantWriter	StructVector	VARIANT (含 Shredding 支持)
ArrayWriter	ListVector	ARRAY
VectorWriter	FixedSizeListVector	VECTOR
MapWriter	MapVector	MAP
RowWriter	StructVector	ROW

2.2.2 转换器继承体系

类名	继承关系	职责
ArrowBatchConverter	抽象基类	管理可复用 VectorSchemaRoot 和 ArrowFieldWriter[]
ArrowPerRowBatchConverter	extends ArrowBatchConverter	逐行迭代写入，适用于非向量化场景
ArrowVectorizedBatchConverter	extends ArrowBatchConverter	向量化批量写入，支持删除向量过滤

2.3 外部依赖关系

paimon-arrow 模块实现了以下来自 paimon-core/paimon-common 的接口：

模块内类	实现/扩展	外部接口	来源模块
ArrowBundleRecords	implements	BundleRecords	paimon-core (io)
ArrowBundleWriter	implements	BundleFormatWriter	paimon-core (format)

模块内类	实现/扩展	外部接口	来源模块
ArrowFieldTypeVisitor	implements	DataTypeVisitor<FieldType>	paimon-common (types)
Arrow2PaimonVectorConverterVisitor	implements	DataTypeVisitor<Converter>	paimon-common (types)
ArrowFieldWriterFactoryVisitor	implements	DataTypeVisitor<Factory>	paimon-common (types)

第3章 核心类设计模式详解

3.1 Visitor 模式 — DataTypeVisitor 体系

paimon-arrow 模块大量使用了 Visitor（访问者）设计模式，通过实现 Paimon 核心的 DataTypeVisitor 接口，对不同数据类型执行差异化处理。模块内有 3 个核心 Visitor 实现：

- ArrowFieldTypeVisitor：将 Paimon DataType 转换为 Arrow FieldType，每个 visit 方法返回对应的 Arrow 类型（如 BooleanType → BIT，DecimalType → Decimal128）
- ArrowFieldWriterFactoryVisitor：为每种 Paimon 类型创建对应的 ArrowFieldWriterFactory，工厂负责实例化具体的 ArrowFieldWriter 子类
- Arrow2PaimonVectorConverterVisitor：为每种类型创建从 Arrow FieldVector 到 Paimon ColumnVector 的转换器

设计意图：通过 Visitor 模式将类型分派逻辑集中管理，新增类型时只需在 Visitor 中增加 visit 方法，无需修改已有代码，符合开闭原则。

3.2 Abstract Factory 模式 — ArrowFieldWriterFactory

ArrowFieldWriterFactory 是一个函数式接口（@FunctionalInterface），定义了 create(FieldVector, boolean) 方法。ArrowFieldWriterFactoryVisitor 充当抽象工厂的角色，根据数据类型返回不同的工厂实例。

关键方法：

- ArrowFieldWriterFactory.create() — 根据 FieldVector 和 nullable 标志创建具体 ArrowFieldWriter
- ArrowFieldWriterFactoryVisitor.visit() — 每个 visit 方法返回方法引用或 Lambda 表达式作为工厂

设计意图：将写入器的创建逻辑与使用逻辑分离，支持灵活的自定义扩展（如 CustomDecimalArrowConversion 测试类展示了自定义 Decimal 写入工厂的实现）。

3.3 Template Method 模式 — ArrowFieldWriter / ArrowBatchConverter

ArrowFieldWriter 抽象类使用了 Template Method（模板方法）模式：

- write(rowIndex, getters, pos)：模板方法，先检查 null，再调用抽象方法 doWrite()
- write(columnVector, pickedInColumn, startIndex, batchRows)：模板方法，调用 doWrite() 后设置 valueCount
- doWrite()：由子类实现的具体写入逻辑

同样，ArrowBatchConverter 使用模板方法 next() 定义了「reset → doWrite → return root」的流程骨架，ArrowPerRowBatchConverter 和 ArrowVectorizedBatchConverter 各自实现 doWrite() 的具体策略。

3.4 Strategy 模式 — 写入策略切换

ArrowBatchConverter 的两个子类体现了 Strategy（策略）模式：

- ArrowPerRowBatchConverter：逐行迭代策略，适用于普通 RecordIterator
- ArrowVectorizedBatchConverter：向量化批量策略，直接操作 VectorizedColumnBatch，支持删除向量过滤（DeletionVectorJudger）

两种策略通过 copy() 方法支持运行时复制，使得上层可以根据数据源类型动态选择最优策略。

3.5 Adapter 模式 — Arrow2PaimonVectorConverter

Arrow2PaimonVectorConverter 接口及其内部 Visitor 实现了 Adapter（适配器）模式，将 Apache Arrow 的 FieldVector 接口适配为 Paimon 的 ColumnVector 接口。每个 visit 方法返回一个匿名内部类，将 Arrow 的 get/isNull 调用适配为 Paimon ColumnVector 的对应方法。

3.6 Facade 模式 — ArrowUtils

ArrowUtils 是一个工具类 Facade（门面），封装了创建 VectorSchemaRoot、FieldVector、ArrowFieldWriter[]、序列化到 C 结构体和 IPC 等常用操作的复杂性，为上层提供简洁的静态方法入口。

第4章 核心流程图

4.1 InternalRow 写入 Arrow VectorSchemaRoot 流程

该流程描述了将 Paimon InternalRow 数据逐行写入 Arrow VectorSchemaRoot 的完整路径：

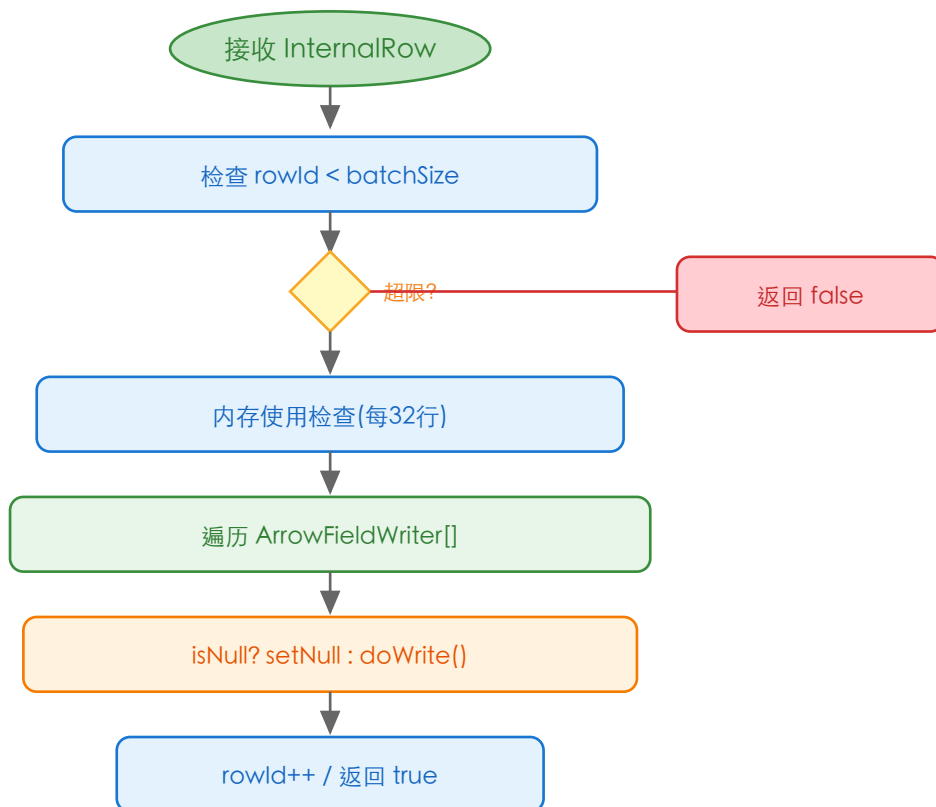


图4-1 ArrowFormatWriter.write() 行写入流程图

4.2 向量化批量转换流程 (ArrowVectorizedBatchConverter)

该流程描述了从 VectorizedRecordIterator 或 DeletionFileRecordIterator 转换到 Arrow 格式的过程：

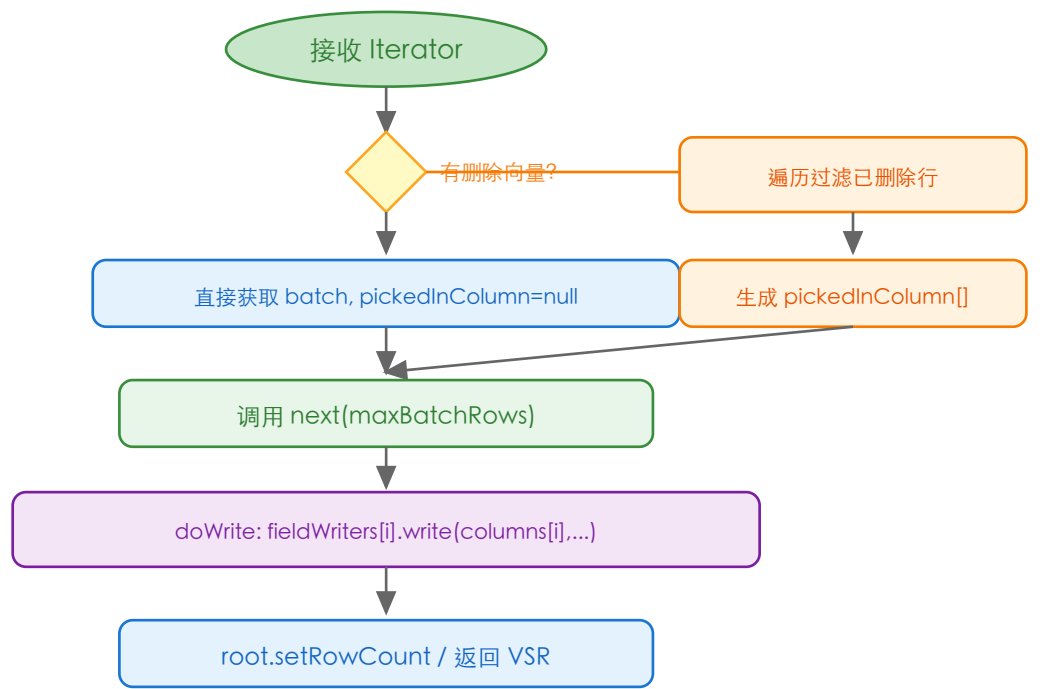


图4-2 ArrowVectorizedBatchConverter 向量化转换流程图

4.3 Arrow C Data Interface 序列化流程

ArrowBundleWriter 通过 Arrow C Data Interface 将数据传输到 Native 层的完整流程：

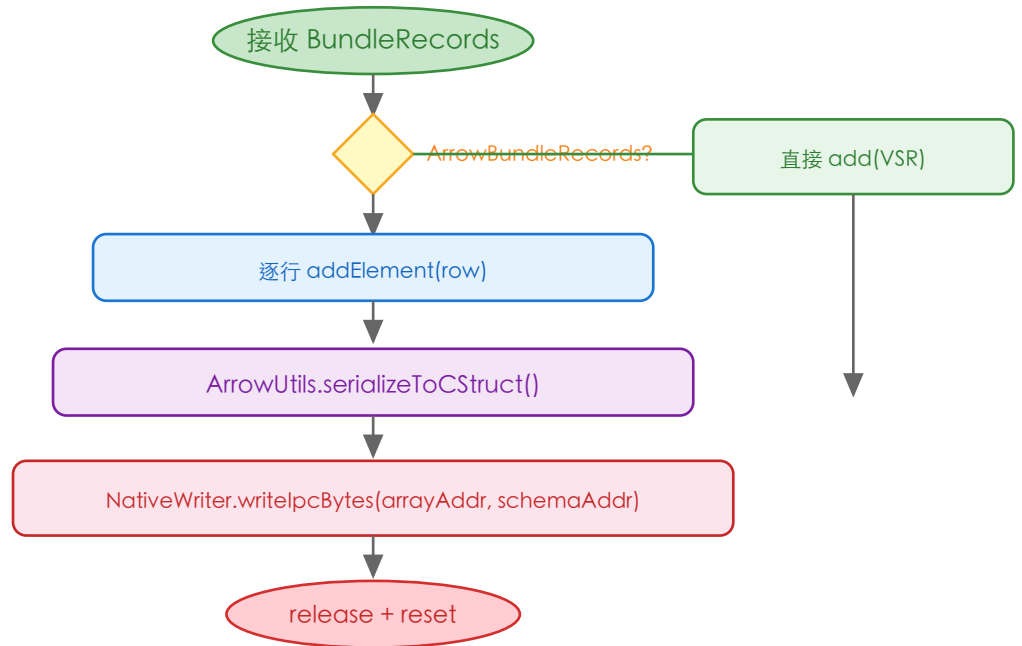


图4-3 Arrow C Data Interface 序列化与 JNI 传输流程图

第5章 核心时序图

5.1 ArrowFormatWriter 写入与读取回环时序图

描述从 InternalRow 写入到 ArrowFormatWriter，再通过 ArrowBatchReader 读取回 InternalRow 的完整交互过程：

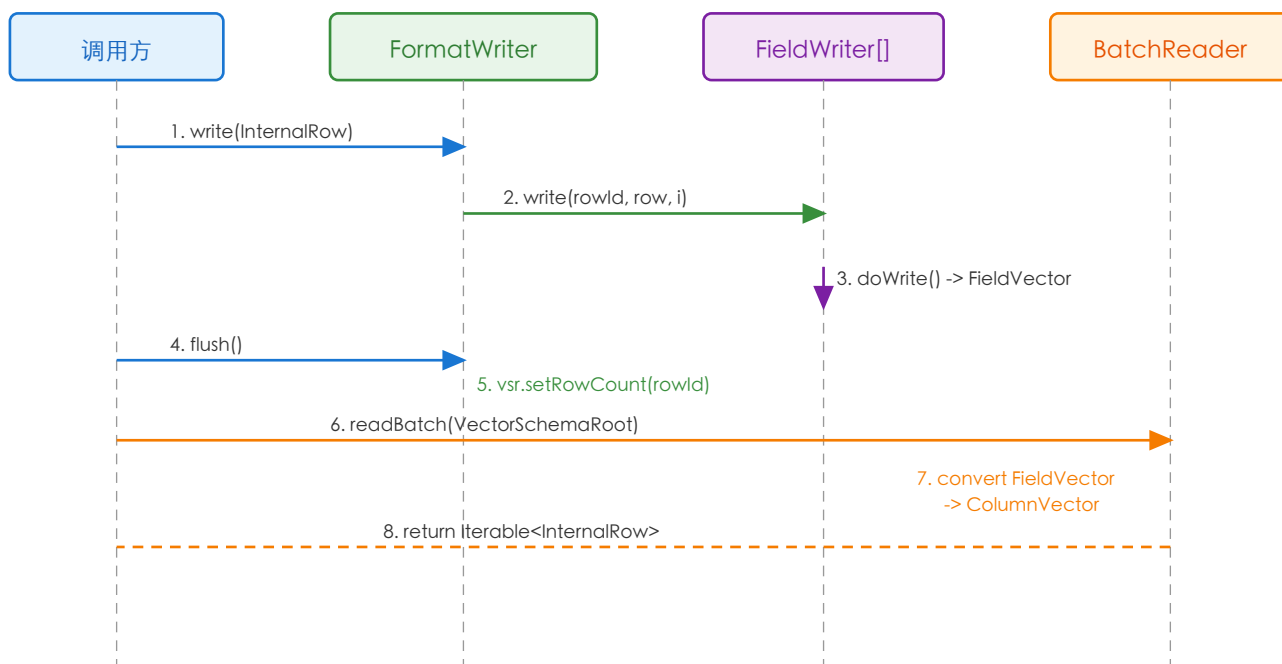


图5-1 ArrowFormatWriter 写入与 ArrowBatchReader 读取时序图

5.2 ArrowBundleWriter 写入到 Native 层时序图

描述 ArrowBundleWriter 如何通过 ArrowFormatCWriter 和 NativeWriter 将数据最终传输到 Native 代码：

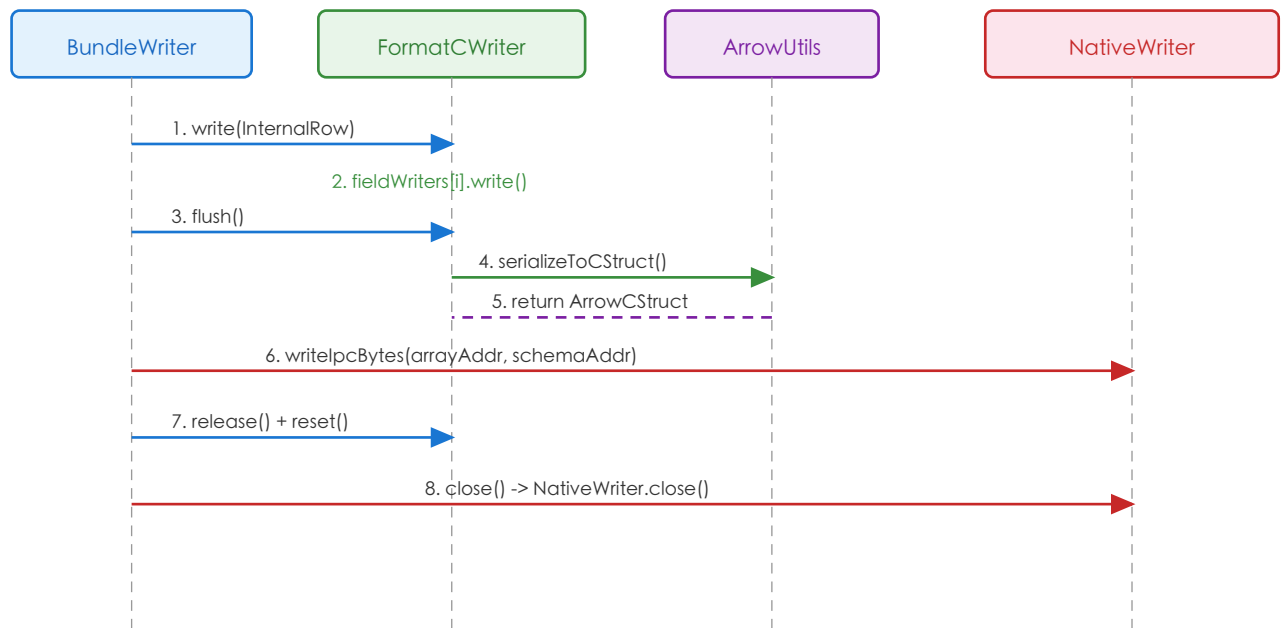


图5-2 ArrowBundleWriter 写入到 Native 层的完整时序图

第6章 关键场景调用链

6.1 场景一：InternalRow 逐行写入 Arrow 格式

触发条件：上层系统（如 Paimon-Python、Paimon-Lance）调用 ArrowFormatWriter.write(InternalRow) 写入数据

步骤	类.方法	说明
1	ArrowFormatWriter.write(InternalRow)	入口方法，检查 batchSize 和内存限制
2	ArrowFieldWriter.write(rowId, row, i)	模板方法，null 检查后调度到 doWrite()
3	ArrowFieldWriters.XXXWriter.doWrite()	具体写入逻辑，调用 Arrow Vector 的 setSafe()
4	ArrowFormatWriter.flush()	设置 VectorSchemaRoot 的 rowCount
5	ArrowFormatWriter.reset()	重置所有 fieldWriter 和 rowId

6.2 场景二：向量化批量读取并转换到 Arrow 格式

触发条件：Paimon 查询引擎以向量化方式读取数据并需要转换为 Arrow 格式

步骤	类.方法	说明
1	ArrowVectorizedBatchConverter.reset(VectorizedRecordIterator)	初始化批次，获取 VectorizedColumnBatch
2	ArrowBatchConverter.next(maxBatchRows)	模板方法，先 reset 所有 writer，再调用 doWrite
3	ArrowVectorizedBatchConverter.doWrite()	计算实际写入行数，调用 fieldWriters[i].write(columns[i], ...)
4	ArrowFieldWriter.write(ColumnVector, picked, start, rows)	向量化写入，遍历 ColumnVector 批量设置
5	root.setRowCount(batchRows)	设置输出的 VectorSchemaRoot 行数

6.3 场景三：带删除向量的向量化转换

触发条件：读取含删除向量（Deletion Vector）的数据文件

步骤	类.方法	说明
1	ArrowVectorizedBatchConverter.reset(DeletionFileRecordIterator)	初始化，获取内部 VectorizedRecordIterator
2	DeletionVectorJudger.isDeleted(position)	遍历每行检查是否被删除
3	IntArrayList.add(i)	收集未删除行的索引到 pickedInColumn[]
4	ArrowFieldWriter.write(col, pickedInColumn, 0, batchRows)	仅写入 picked 的行，跳过已删除数据
5	getRowNumber(startIndex, i, pickedInColumn)	通过间接寻址定位原始列向量中的行号

6.4 场景四：Arrow VectorSchemaRoot 反向读取为 Paimon 行

触发条件：接收来自 Arrow 格式的数据（如跨语言调用），需转换回 Paimon InternalRow

步骤	类.方法	说明
1	ArrowBatchReader.readBatch(VectorSchemaRoot)	入口，建立字段名到 Arrow Schema 的映射
2	Arrow2PaimonVectorConverter.construct(dataType)	通过 Visitor 为每种类型创建转换器
3	converter.convertVector(fieldVector)	将 Arrow FieldVector 适配为 Paimon ColumnVector
4	VectorizedColumnBatch.setNumRows()	设置批次行数
5	ColumnarRow.setRowId(position)	通过 ColumnarRow 包装，返回 Iterator

6.5 场景五：C Data Interface 序列化传输

触发条件：需要通过 JNI 将 Arrow 数据传递给 C/C++/Rust Native 代码

步骤	类.方法	说明
1	ArrowFormatCWriter.write(InternalRow)	委托给内部 ArrowFormatWriter 写入
2	ArrowFormatCWriter.flush()	调用 realWriter.flush() 设置行数
3	ArrowFormatCWriter.toCStruct()	调用 ArrowUtils.serializeToCStruct()
4	Data.exportVectorSchemaRoot()	Apache Arrow Java 库导出为 C 结构体
5	ArrowCStruct.arrayAddress() / schemaAddress()	获取内存地址传递给 Native 代码

6.6 场景六：Variant Shredding 写入

触发条件：写入包含 Variant 类型且配置了 Shredding Schema 的数据

步骤	类.方法	说明
1	ArrowFormatWriter 构造函数	检测 VariantType，加载 ShreddingSchema
2	ArrowFieldWriters.VariantWriter 构造	根据 shreddingSchema 构建 VariantSchema 和子 Writer
3	VariantWriter.doWrite(rowIndex, getters, pos)	获取 Variant，判断是否需要 Shredding
4	PaimonShreddingUtils.castShredded(variant, schema)	将 Variant 拆解为结构化行
5	writeRow(rowIndex, shreddedRow)	遍历子 Writer 写入拆解后的各字段

第7章 配置与扩展

7.1 配置参数

参数	类型	说明
writeBatchSize	int	每批次最大写入行数，控制 VectorSchemaRoot 的容量
caseSensitive	boolean	字段名是否大小写敏感，影响 Arrow Schema 字段命名
memoryUsedMaxInBytes	Long (nullable)	Arrow 内存使用上限（字节），超限时 write() 返回 false
shreddingSchemas	RowType (nullable)	Variant Shredding 的目标 Schema，null 表示不启用

7.2 扩展点

paimon-arrow 模块提供了多个可扩展的抽象点：

- ArrowFieldTypeConversion.ArrowFieldTypeVisitor：可继承并重写 visit() 方法自定义类型映射（如将 Decimal 映射为 Binary，参见 CustomDecimalArrowConversion 测试类）
- ArrowFieldWriterFactoryVisitor：可继承并重写 visit() 方法自定义 Writer 工厂（支持自定义序列化格式）
- Arrow2PaimonVectorConvertorVisitor：可继承并重写 visit() 方法自定义反向转换逻辑
- NativeWriter / NativeReader：抽象类，由具体的 Native 实现（如 paimon-lance）提供 JNI 绑定
- ArrowBatchConverter.copy()：支持转换器的复制，便于并行处理场景

第8章 设计亮点与总结

8.1 设计亮点

- 全类型覆盖的 Visitor 体系：通过 DataTypeVisitor 模式实现了 Paimon 全部 20+ 数据类型与 Arrow 类型的完整映射，包括复杂嵌套类型（Array、Map、Row、Vector）和新型 Variant 类型
- 双模式写入策略：ArrowPerRowBatchConverter（逐行）和 ArrowVectorizedBatchConverter（向量化批量）两种策略可根据数据源特性灵活切换，向量化路径避免了行级拆装开销
- 删除向量无缝集成：ArrowVectorizedBatchConverter 通过 pickedInColumn 间接寻址机制，无需物理删除即可跳过已删除行，与 Paimon 的 Deletion Vector 特性完美配合
- Variant Shredding 支持：VariantWriter 支持将半结构化 Variant 数据拆解为结构化 Arrow Schema，实现列式存储和查询优化
- 内存安全机制：ArrowFormatWriter 支持可选的内存使用上限检查（每 32 行检测一次），防止 OOM；对 OversizedAllocationException 和 IndexOutOfBoundsException 有优雅降级处理
- 高性能字符串写入优化：StringWriter 的 doWrite() 方法对单 MemorySegment 的 BinaryString 进行零拷贝写入（直接使用 heapMemory + offset），避免不必要的字节数组复制
- 可扩展的工厂体系：通过 ArrowFieldWriterFactory 函数式接口和 Visitor 的组合，支持外部自定义类型转换（如 CustomDecimalArrowConversion 展示的 Decimal→Binary 自定义映射）

8.2 设计限制

- MultisetType 不支持：ArrowFieldTypeVisitor 和 ArrowFieldWriterFactoryVisitor 对 MultisetType 均抛出 UnsupportedOperationException
- BlobType 不支持：Blob 类型在类型转换和向量转换中均不支持
- NativeWriter/NativeReader 为空壳：抽象类仅定义了接口，实际实现依赖外部模块（如 paimon-lance）通过 JNI 提供
- 调试日志硬编码：ArrowBundleWriter.close() 中使用了 System.out.println 输出性能统计，应改为 LOG.info

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	四层架构清晰，读写分离，职责单一
设计模式运用	★★★★★	Visitor、Template Method、Strategy、Factory、Adapter、Facade 六种模式综合运用
可扩展性	★★★★★	三个核心 Visitor 均支持继承扩展，NativeWriter/NativeReader 提供 Native 扩展点

评估维度	评分	说明
类型完整性	★★★★☆	覆盖全部常用类型和 Variant Shredding，但 Multiset/Blob 不支持
性能优化	★★★★★	向量化写入、零拷贝字符串、内存限制检查、删除向量间接寻址等多维度优化
文档完整度	★★★★☆☆	代码注释较少，类级 Javadoc 简洁，方法级文档不足